

Unit 01: Principles of OOPs and basics of C++

CONTENTS

Objectives

Introduction

- 1.1 Basic Concept of Object-oriented Programming
- 1.2 Introduction to OOP languages
- 1.3 Procedural programming v/s Object-oriented programming
- 1.4 Procedure oriented Programming Paradigm
- 1.5 Object-oriented Programming Paradigm
- 1.6 Benefits of OOP
- 1.7 Applications of Object Oriented Programming
- 1.8 C++ Class Member Function
- 1.9 Private Member Function

Summary

Keywords

Self Assessment

Answers for Self Assessment

Review Questions

Further Readings

Objectives

After studying this unit, you will be able to:

- Recognize the basic concept of object-oriented programming
- Describe the OOP languages
- Understand Basics of C++
- Compare the procedural Oriented and object-oriented programming

Introduction

Over the last few decades, programming practices have changed dramatically. As more programmers gained competence, previously undiscovered difficulties began to emerge. The programming community became increasingly worried about the programming philosophy they use and the methodologies they use in software development.

Productivity, reliability, cost effectiveness, reusability, and other factors began to become key concerns. Many conscious attempts were made to comprehend these issues and find potential answers. This is precisely why a growing number of programming languages have been developed and are still being developed. Furthermore, techniques to programme creation have been the subject of extensive research, resulting in the development of several frameworks. The object-oriented programming method, or simply OOP, is one such approach, and it is perhaps the most common.

C++ programming's main goal is to introduce the concept of object orientation to the C programming language.

Inheritance, data binding, polymorphism, and other notions are all part of the Object Oriented Programming paradigm.

1.1 Basic Concept of Object-oriented Programming

Object oriented programming is a type of programming which uses objects and classes its functioning. The object oriented programming is based on real world entities like inheritance, polymorphism, data hiding, etc. It aims at binding together data and function work on these data sets into a single entity to restrict their usage.

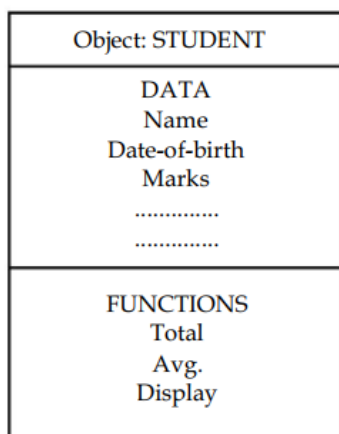
In object-oriented parlance, a problem is viewed in terms of the following concepts:

1. Objects
2. Classes
3. Data abstraction
4. Data encapsulation
5. Inheritance
6. Polymorphism
7. Dynamic binding
8. Message passing

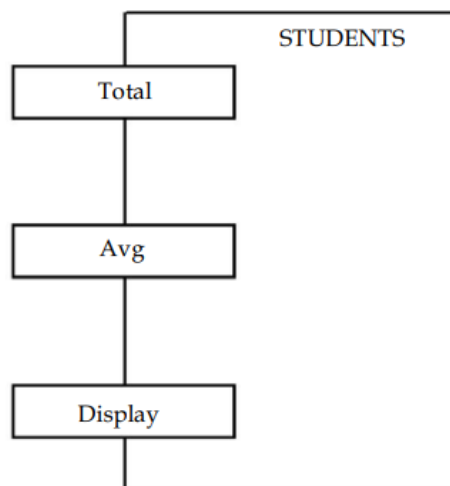
Let us now study the entire concept in detail.

1. **Objects:** Objects are the basic run-time entities in an object-oriented system. They may represent a person, a place, a bank account, a table of data; they may also represent user-defined data such as vectors, time and lists.

They occupy space in memory that keeps its state and is operated on by the defined operations on the object. Each object contains data and code to manipulate the data.



(a)



(b)



Notes: -Objects can interact without having to know details of each other data or code.

2. **Classes:** A class represents a set of related objects. The object has some attributes, whose value consist much of the state of an object. The class of an object defines what attributes an object has. The entire set of data and code of an object can be made a user-defined data type with the help of a class.

Classes are user defined data types that behave like the built-in types of a programming language. Classes have an interface that defines which part of an object of a class can be accessed from outside and how. A class body that implements the operations in the interface, and the instance variables that contain the state of an object of that class.



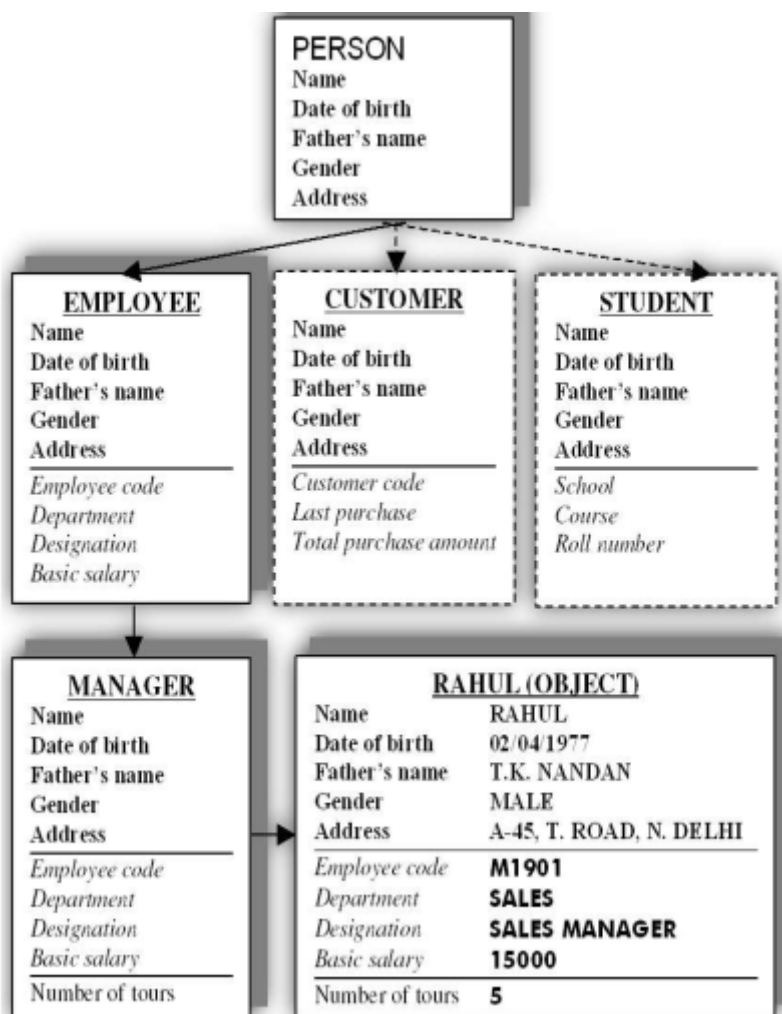
Notes:- A class is a data-type that has its own members i.e. data members and member functions. It is the blueprint for an object in objects oriented programming language. It is the basic building block of object oriented programming in c++.

The data and the operation of a class can be declared as one of the three types:

- (a) **Public:** These are declarations that are accessible from outside the class to anyone who can access an object of this class.
- (b) **Protected:** These are declarations that are accessible from outside the class to anyone who can access an object of this class.
- (c) **Private:** These are declarations that are accessible only from within the class itself.

Syntax of class

```
class class_name {
    data_type data_name;
    return_type method_name(parameters);
}
```



3. **Data abstraction:** - Data abstraction or Data Hiding is the concept of hiding data and showing only relevant data to the final user. It is also an important part object oriented programming.

Let's take real life example to understand concept better, when we ride a bike we only know that pressing the brake will stop the bike and rotating the throttle will accelerate but you don't know how it works and it is also not think we should know that's why this is done from the same as a concept data abstraction.

In C plus plus programming language write two ways using which we can accomplish data abstraction –

1. using class
2. using header file



Example: - We can represents essential features without including background details and explanations.

index of text book.

```
class School
{
void sixthc lass();
void seventhclass();
void tenthclass();
}
```

4. **Data Encapsulation:** In object oriented programming, encapsulation is the concept of wrapping together of data and information in a single unit. A formal definition of encapsulation would be: encapsulation is binding together the data and related function that can manipulate the data.
5. **Inheritance:** A class's capacity to inherit or derive attributes or characteristics from other classes is known as inheritance. It is particularly significant in an object-oriented software since it allows for reusability, i.e. using a method defined in another class via inheritance. Child class or subclass is a class that inherits properties from another class, while base class or parent class is the class from which the properties are inherited.



Notes: - Inheritance allows us to create a new class (derived class) from an existing class (base class).

C plus plus programming language supports the following types of inheritance

- single inheritance
- multiple inheritance
- multi level inheritance
- Hierarchical inheritance
- hybrid inheritance



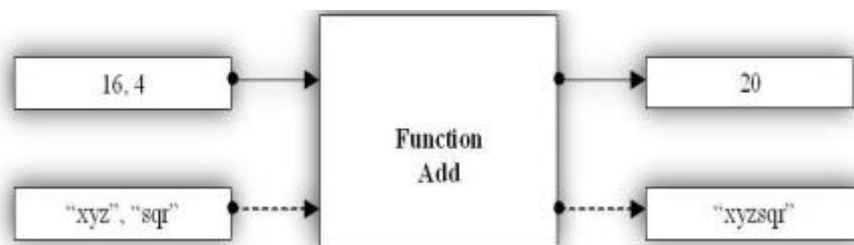
Caution: - Keep in mind that each subclass defines only those features that are unique to it.

6. Polymorphism:-

Polymorphism means the ability to take more than one form. An operation may exhibit different behavior in different instances. The behavior depends upon the types of the data used in the operation. For example considering the operator plus (+).

$16 + 4 = 20$

"xyz" + "sqr" = "xyzsqr"

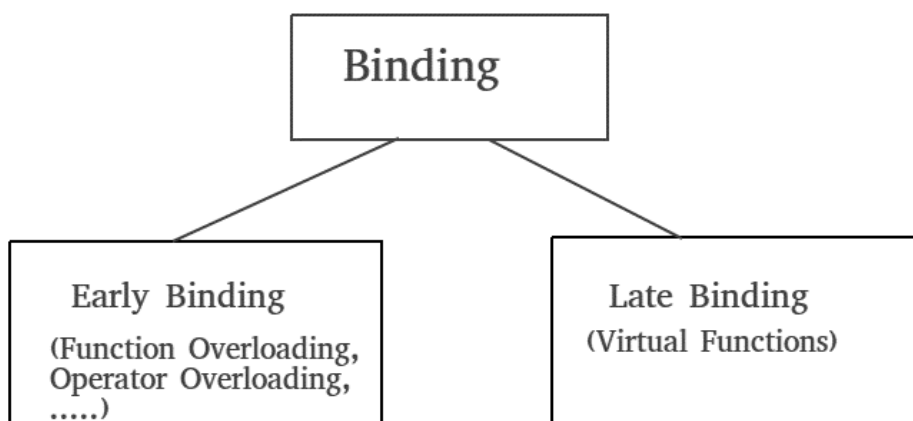


Example: - A person can have more than one behavior depending upon the situation. like a woman a mother, manager and a daughter And this define her behavior. This is from where the concept of polymorphism came from.

In c++ programming language, polymorphism is achieved using two ways. They are operator overloading and function overloading.

7. Dynamic Binding:-

Binding refers to the process of converting identifiers (such as variable and performance names) into addresses. Binding is done for each variable and functions. For functions, it means that matching the call with the right function definition by the compiler. It takes place either at compile time or at runtime.



This is associated with polymorphism and inheritance. A function call associated with a polymorphic reference depends on the dynamic type of that reference. For example in the above figure, by inheritance, every object will have this procedure. Its algorithm is, however, unique to

each object so the procedure will be redefined in each class that defines the objects. At run-time, the code matching the object under reference will be called.



Difference between static and dynamic binding in C++

Did you know?

BASIS FOR COMPARISON	STATIC BINDING	DYNAMIC BINDING
Event Occurrence	Events occur at compile time are "Static Binding".	Event Occurrence
Information	All information needed to call a function is known at compile time.	All information need to call a function come to know at run time.
Advantage	Efficiency.	Flexibility.
Time	Fast execution.	Slow execution.
Alternate name	Early Binding.	Late Binding.

8. Message Passing

Objects communicate with one another by sending and receiving information to each other. A message for an object is a request for execution of a procedure and therefore will invoke a function in the receiving object that generates the desired results. Message passing involves specifying the name of the object, the name of the function and the information to be sent.



Example

```
#include<iostream>
using namespace std;
class demo{
public:
int f_num,s_num;
sum(int a,int b){
cout<<a+b;
}
};
main(){
demo d1;
d1.sum(d1.f_num=10,d1.s_num=39);
return 0;
}
```

1.2 Introduction to OOP languages

Object oriented programming is not the right of any particular language. Although languages like C and Pascal can be used but programming becomes clumsy and may generate confusion when

program grow in size. A language that is specially designed to support the OOP concepts makes it easier to implement them.

To claim that they are object-oriented they should support several concepts of OOP.

Depending upon the features they support, they are classified into the following categories:

1. Object-based programming languages.
2. Object-oriented programming languages.

Characteristics	Sim ula	Small talk 80	Objective C	C ++	ADA	Object Pascal	Eittel
Binding (early or late)	Both	Late	Both	Both	Early	Late	Early
polymorphism (operator overloading)	Yes	Yes	Yes	Yes	Yes	Yes	Yes
Data hiding	Yes	Yes	Yes	Yes	Yes	Yes	Yes
Concurrency	Yes	Poor	Poor	Poor	Difficult	No	Promised
Inheritance	Yes	Yes	Yes	Yes	No	Yes	Yes
Multiple Inheritance	No	Yes	Yes	Yes	No	Yes	Yes
Garbage Collection	Yes	Yes	Yes	Yes	No	Yes	Yes
Persistence	No	Pormised	No	No	Like 3GL	No	Some support
Genericity	No	No	No	No	Yes	No	Yes
Object lib	Yes	Yes	Yes	No	Not much	Yes	yes

Major features required by object-based programming are:

1. Polymorphism
2. Data encapsulation
3. Data hiding
4. Operator Overloading
5. Inheritance

Languages that support programming with objects are said to be object-based programming languages. These do not support inheritance and dynamic binding ADA is a typical example Object oriented programming incorporates all of objects-based programming features along with two additional feature, namely, inheritance and dynamic binding.

Languages that support these features:-

C++

.NET

Java

PHP

C#

Python

Ruby

1.3 Procedural programming v/s Object-oriented programming

Let's see the comparison between Procedural programming and object-oriented programming. We are comparing both terms on the basis of some characteristics. The difference between both languages are tabulated as follows –

On the basis of	Procedural Programming	Object-oriented programming
Definition	It is a programming language that is derived from structure programming and based upon the concept of calling procedures. It follows a step-by-step approach in order to break down a task into a set of variables and routines via a sequence of instructions.	Object-oriented programming is a computer programming design philosophy or methodology that organizes/ models software design around data or objects rather than functions and logic.
Approach	It follows a top-down approach.	It follows a bottom-up approach.
Importance	It gives importance to functions over data.	It gives importance to data over functions.
Data hiding	There is not any proper way for data hiding.	There is a possibility of data hiding.
Program division	In Procedural programming, a program is divided into small programs that are referred to as functions.	In OOP, a program is divided into small parts that are referred to as objects.

In the modern programming parlance, at least in most of the commercial and business applications areas, programming has been made independent of the target machine. This machine independent characteristic of programming has given rise to a number of different methodologies in which programs can now be developed. We will particularly concern ourselves with two broad programming approaches – or paradigm as they are called in the present context.

1. Procedure-oriented paradigm
2. Object-oriented paradigm

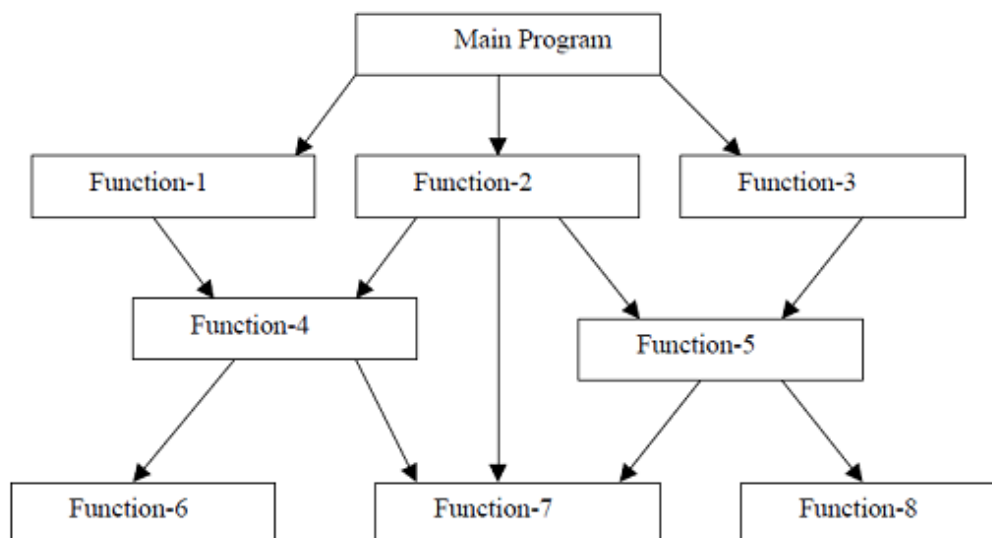
1.4 Procedure oriented Programming Paradigm

Before you get into OOP, take a look at conventional procedure-oriented programming in a language such as C. Using the procedure-oriented approach; you view a problem as a sequence of things to do such as reading, calculating and printing. Conventional programming using high-level languages is commonly known as procedure-oriented programming.



Example C, COBOL and FORTRAN

You organize the related data items into C structures and write the necessary functions (procedures) to manipulate the data and, in the process, complete the sequence of tasks that solve your problem. Structure of procedure oriented paradigm is shown in following figure.



Many key data items are put as global in a multi-function software so that they can be accessible by all functions. It's possible that each function has its own set of local data. Global data are more vulnerable to a function's unintended alteration. It's difficult to figure out which data is used by which function in a huge software. If we need to make changes to an external data structure, we must likewise make changes to any functions that access the data. Bugs will be able to get in through this opening.

Another serious drawback with the procedural approach is that we do not model real world problems very well. This is because functions are action-oriented and do not really corresponding to the element of the problem.

Some Characteristics exhibited by procedure-oriented programming are:-

Emphasis is on doing things (algorithms).

Large programs are divided into smaller programs known as functions.

Most of the functions share global data.

Data move openly around the system from function to function.

Functions transform data from one form to another.

1.5 Object-oriented Programming Paradigm

The term Object-oriented Programming (OOP) is widely used, but experts do not seem to agree on its exact definition. However, most experts agree that OOP involves defining Abstract Data Types (ADT) representing complex real-world or abstract objects and organizing programs around the collection of ADTs with an eye toward exploiting their common features. The term data abstraction refers to the process of defining ADTs; inheritance and polymorphism refer to the mechanisms that enable you to take advantage of the common characteristics of the ADTs – the objects in OOP.

Before going any further into OOP, take note of two points. First, OOP is only a method of designing and implementing software. Use of object-oriented techniques does not impart anything to a finished software product that the user can see. However, as a programmer while implementing the software, you can gain significant advantages by using object-oriented methods, especially in large software projects. Because OOP enables you to remain close to the conceptual, higher-level model of the real-world problem you are trying to solve, you can manage the complexity better than with approaches that force you to map the problem to fit the features of the language. You can take advantage of the modularity of objects and implement the program in

relatively independent units that are easier to maintain and extend. You can also share code among objects through inheritance.

Secondly, OOP has nothing to do with any programming language, although a programming language that supports OOP makes it easier to implement the object-oriented techniques. As you will see shortly, with some discipline, you can use objects even in C programs.

1.6 Benefits of OOP

- It is easy to model a real system as real objects are represented by programming objects in OOP. The objects are processed by their member data and functions. It is easy to analyze the user requirements.
- With the help of inheritance, we can reuse the existing class to derive a new class such that the redundant code is eliminated, and the use of existing class is extended. This saves time and cost of program.
- Modular approach is used for write code.
- In OOP, data can be made private to a class such that only member functions of the class can access the data. This principle of data hiding helps the programmer to build a secure program that cannot be invaded by code in other part of the program.
- It is very easy to partition the work in a project based on objects.
- It is possible to map the objects in problem domain to those in the program.
- With the help of polymorphism, the same function or same operator can be used for different purposes. This helps to manage software complexity easily.
- Large problems can be reduced to smaller and more manageable problems. It is easy to partition the work in a project based on objects.

1.7 Applications of Object Oriented Programming

Perhaps the most logical description of the real world is the object-oriented approach. As a result, it may be used in practically every problem-solving circumstance. OOP has been widely accepted in the software industry due to its effectiveness in problem solving and programming. Existing systems will be converted to OOP.

The framework, which spans all phases of system development, is one fundamental component of OOP that is extremely advantageous. Thus, OOA (Object Oriented Analysis), OOD (Object Oriented Design), OOT (Object Oriented Testing), and other object-oriented tools are significantly more appropriate than non-object-oriented ones.

Object oriented programming provides many applications:

- **Client-Server Systems:** Object-oriented client-server systems provide the IT infrastructure, creating Object-Oriented Client-Server Internet (OCSI) applications. Here, infrastructure refers to operating systems, networks, and hardware. OSCI consist of three major technologies:
 - The Client Server
 - Object-Oriented Programming
 - The Internet
- **Real-Time System:** A real time system is a system that give output at given instant and its parameters changes at every time. A real time system is nothing but a dynamic system. Dynamic means the system that changes every moment based on input to the system. OOP approach is very useful for Real time system because code changing is very easy in OOP system and it leads toward dynamic behavior of OOP codes thus more suitable to real time system.

- **Object-Oriented Databases:** They are also called Object Database Management Systems (ODBMS). These databases store objects instead of data, such as real numbers and integers. Objects consist of the following:
 - **Attributes:** Attributes are data that define the traits of an object. This data can be as simple as integers and real numbers. It can also be a reference to a complex object.
 - **Methods:** They define the behavior and are also called functions or procedures.
- **Simulation and modelling:** Another area where OOP approach criteria might be counted is system modelling. Because OOP programmers are easier to grasp, it is preferable to represent a system in a simpler form when using the OOP approach.
- **AI and expert systems:** These are computer applications that are developed to solve complex problems pertaining to a specific domain, which is at a level far beyond the reach of a human brain.
 - It has the following characteristics:
 - Reliable
 - Highly responsive
 - Understandable
 - High-performance
- **Neural networks and parallel programming:** It address the problem of prediction and approximation of complex time-varying systems. Firstly, the entire time-varying process is split into several time intervals or slots. Then, neural networks are developed in a particular time interval to disperse the load of various networks. OOP simplifies the entire process by simplifying the approximation and prediction ability of networks.

1.8 C++ Class Member Function

Member function of a class is a function that has its definition or its prototype within the class definition like any other variable. It operates on any object of the class.

A member function is defined outside the class using the::(double colon symbol) scope resolution operator. This is useful when we did not want to define the function within the main program, which makes the program more understandable and easier to maintain.

Syntax

```
return_type class_name :: member_function
```



Example

```
#include<iostream>
using namespace std;
class find_sum{
public:
    int x,y;
    int sum();
}
```

```
};  
int find_sum ::sum(){  
    return x+y;  
}
```

1.9 Private Member Function

A private member variable or function cannot be accessed, or even viewed from outside the class. Only the class and friend functions can access private members.



Example

```
#include <iostream>  
  
using namespace std;  
  
class Box {  
    public:  
        double length;  
        void setWidth( double wid );  
        double getWidth( void );  
    private:  
        double width;  
};  
  
double Box::getWidth(void) {  
    return width ;  
}
```

Summary

- Programming practices have evolved considerably over the past few decades.
- By the end of last decade, millions and millions of lines of codes have been designed and implemented all over the world.
- The main objective is to reuse these lines of codes. More and more software development projects were software crisis.
- It is this immediate crisis that necessitated the development of object-oriented approach which supports reusability of the existing code.
- Software is not manufactured. It is evolved or developed after passing through various developmental phases including study, analysis, design, implementation, and maintenance.
- Conventional programming using high level languages such as COBOL, FORTRAN and C is commonly known as procedures-oriented programming.
- In order to solve a problem, a hierarchical decomposition has been used to specify the tasks to be completed.
- OOP is a method of designing and implementing software.
- Since OOP enables you to remain close to the conceptual, higher-level model of the real world problem, you can manage the complexity better than with approaches that force you to map the problem to fit the features of the language.

- Some essential concepts that make a programming approach object-oriented are objects, classes, Data abstraction, Data encapsulation, Inheritance, Polymorphism, dynamic binding and message passing.
- The data and the operation of a class can be disclosed public, protected or private. OOP provides greater programmer productivity, better quality of software and lesser maintenance cost.

Keywords

Classes: A class represents a set of related objects.

Data Abstraction: Abstraction refers to the act of representing essential-features without including the background details or explanations.

Data Encapsulation: The wrapping up to data and functions into a single unit (class) is known as encapsulation.

Design: The term design describes both a final software system and a process by which it is developed.

Dynamic Binding: Binding refers to the linking of a procedure call to the code to be executed in response to the call.

Inheritance: Inheritance is the process by which objects of one class acquire the properties of objects of another class.

Message Passing: Message passing is another feature of object-oriented programming.

Object-oriented Programming Paradigm: The term object-oriented programming (OOP) is widely used, but experts do not seem to agree on its exact definition.

Objects: Objects are the basic run-time entities in an object-oriented system.

Polymorphism: Polymorphism means the ability to take more than one form.

Self Assessment

1. Which feature of OOPS described the reusability of code?
 - A. Abstraction
 - B. Encapsulation
 - C. Polymorphism
 - D. Inheritance
2. Which of the following is not an OOP?
 - A. Java
 - B. C#
 - C. C++
 - D. C
3. OOP acronym for
 - A. Object of Programming
 - B. Object Original Programming
 - C. Object Oriented Programming
 - D. Operating Original Programming
4. Which feature of OOPS derives the class from another class?
 - A. Inheritance

- B. Data hiding
 - C. Encapsulation
 - D. Polymorphism
5. Which of the following is correct about class?
- A. class can have member functions while structure cannot.
 - B. class data members are public by default while that of structure are private.
 - C. Pointer to structure or classes cannot be declared.
 - D. class data members are private by default while that of structure are public by default.
6. Which of the following is not an access specifier?
- A. Public
 - B. Char
 - C. Private
 - D. Protected
7. Which of the following OOP concept is not true for the C++ programming language?
- A. A class must have member functions
 - B. C++ Program can be easily written without the use of classes
 - C. At least one instance should be declared within the C++ program
 - D. C++ Program must contain at least one class
8. What is the extra feature in classes which was not in the structures?
- A. Member functions
 - B. Data members
 - C. Public access specifier
 - D. Static Data allowed
9. Which operator is used to define a member function outside the class?
- A. *
 - B. ()
 - C. +
 - D. ::
10. Nested member function is
- A. A function that call itself again and again.
 - B. A member function may call another member function within itself.
 - C. Same as Class in the program
 - D. Accessed using * operator
11. Which of the following is syntax of C++ class member function?
- A. class_name,function_name
 - B. return_type class_name :: member_function
 - C. datatype_class_name,function_name
 - D. class_name_function_name
12. Which among the following feature does not come under the concept of OOPS?
- A. Platform independent
 - B. Data binding

- C. Data hiding
- D. Message passing

13. The combination of abstraction of the data and code is viewed in_____.

- A. Inheritance
- B. Class
- C. Object
- D. Interfaces

14. Which is private member functions access scope?

- A. Member functions which can used outside the class
- B. Member functions which can only be used within the class
- C. Member functions which are accessible in derived class
- D. Member functions which can't be accessed inside the class

15. Which syntax among the following shows that a member is private in a class?

- A. private::Name(parameters)
- B. private: function Name(parameters)
- C. private(function Name(parameters))
- D. private function Name(parameters)

Answers for Self Assessment

- | | | | | |
|-------|-------|-------|-------|-------|
| 1. D | 2. D | 3. C | 4. A | 5. D |
| 6. B | 7. D | 8. A | 9. D | 10. B |
| 11. B | 12. A | 13. C | 14. B | 15. D |

Review Questions

1. Discuss basic concepts of C++ in detail.
2. Inheritance is the process by which objects of one class acquire the properties of objects of another class. Analyze.
3. Examine what are the benefits of OOP?
4. Compare what you look for in the problem description when applying object-oriented approach in contrast to the procedural approach. Illustrate with some practical examples.
5. What is OOP? Explain the applications of object oriented programming in detail.
6. Differentiate procedural programming and object oriented programming.
7. Write programs that demonstrate working of classes and objects.



Further Readings

- E. Balagurusamy, Object-oriented Programming through C++, Tata McGraw Hill.
- Herbert Schildt, The Complete Reference – C++, Tata Mc Graw Hill.
- Robert Lafore, Object-oriented Programming in Turbo C++, Galgotia Publications



Web Links

https://en.wikipedia.org/wiki/Object-oriented_programming

www.web-source.net

www.webopedia.com